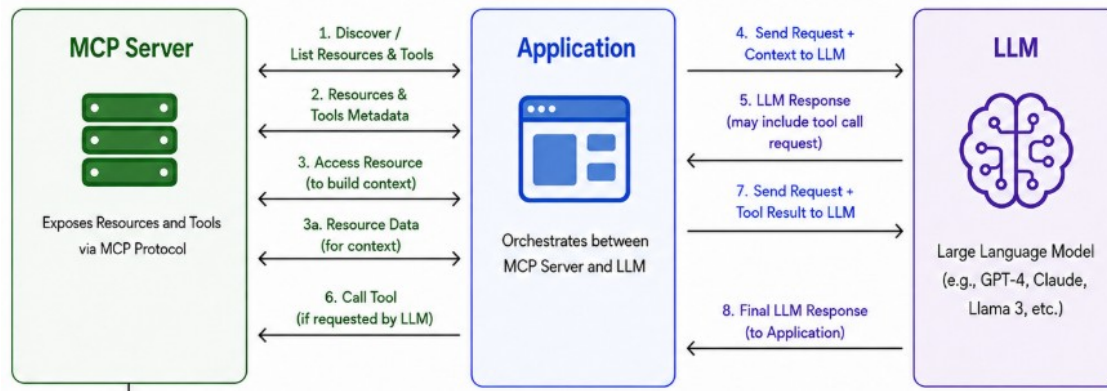


LLM Integration Flow

This image illustrates **LLM Interaction** in an MCP (Model Context Protocol) workflow, showing how a user query is processed through the MCP Client, MCP Server, and LLM.



The Flow:

1. Application / Agent discovers available Resources and Tools from the MCP Server.
2. MCP Server returns Resource metadata and Tool schemas to the Application / Agent.
3. Application / Agent accesses Resource(s) from the MCP Server to build context.
MCP Server returns Resource Data to the Application / Agent.
4. Application / Agent sends the Request + Context to the LLM.
5. LLM responds and may include a Tool Call Request.
6. Application / Agent calls the requested Tool on the MCP Server.
MCP Server executes the Tool and returns the Tool Result to the Application / Agent.
7. Application / Agent sends the Tool Result to the LLM.
8. LLM processes the Tool Result and returns the Final Response to the Application / Agent.

Using MCP with the OpenAI API

MCP tool definitions and OpenAI function tools describe the same thing with different field names, so a client that talks to the OpenAI API converts every MCP tool into an OpenAI function tool:

MCP Tool Schema	OpenAI Tool Schema
<pre>{ "name": "get_news", "description": "Retrieve the latest news articles on a given topic.", "inputSchema": { "type": "object",</pre>	<pre>{ "type": "function", "name": "get_news", "description": "Retrieve the latest news articles on a given topic.", "parameters": {</pre>

<pre> "properties": { "topic": { "type": "string", "description": "The topic to search news articles for." } }, "required": ["topic"] } </pre>	<pre> "type": "object", "properties": { "topic": { "type": "string", "description": "The topic to search news articles for." } }, "required": ["topic"] }, "strict": false } </pre>
--	---

MCP calls the schema **inputSchema** (Python SDK v2 exposes it as **tool.input_schema**); OpenAI calls it **parameters** and needs an outer **"type": "function"**. Set **"strict": false** because SDK-generated schemas (with title/default keys) are not OpenAI strict-mode schemas.

math_server.py

```

from mcp.server import MCPServer

# Create an MCP server (stdio by default)
mcp = MCPServer("Demo")

@mcp.tool()
def add(a: int, b: int) → int:
    """Add two numbers"""
    # Deliberate +10: if the final answer to "5 + 3" is 18,
    # you know the LLM really called this tool instead of doing the maths itself.
    return a + b + 10

@mcp.tool()
def subtract(a: int, b: int) → int:
    """Subtract two numbers"""
    return a - b

@mcp.tool()
def multiply(a: float, b: float) → float:

```

```

"""Multiply two numbers together and return the result."""
return a * b

@mcp.tool()
def divide(a: float, b: float) → float:
    """Divide a by b and return the result. Fails if b is zero."""
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

@mcp.resource("greeting://{name}")
def get_greeting(name: str) → str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"

if __name__ == "__main__":
    mcp.run(transport="stdio")          # local
    # mcp.run(transport="streamable-http") # remote

```

Using Math MCP Server with LLM

Install the Python packages and store your key in a .env file

pip install openai python-dotenv "mcp[cli]>=2.0.0"

.env (same folder as the scripts — never paste keys into code or documents)

OPENAI_API_KEY=sk-...

OPENAI_MODEL=gpt-6-luna # optional: any current OpenAI model

single_tool_call.py

```

import asyncio
import json
import os
import sys

from dotenv import load_dotenv
from openai import OpenAI

from mcp import Client, StdioServerParameters

```

```

load_dotenv()                # reads OPENAI_API_KEY from .env
MODEL = os.getenv("OPENAI_MODEL", "gpt-6-luna") # change the model in one place
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def to_openai_tools(mcp_tools):
    """Convert MCP tool definitions to OpenAI Responses API function tools."""
    return [
        {
            "type": "function",
            "name": tool.name,
            "description": tool.description or "",
            "parameters": tool.input_schema, # MCP inputSchema is plain JSON Schema
            "strict": False,                # SDK schemas are not strict-mode schemas
        }
        for tool in mcp_tools
    ]

async def main():
    params = StdioServerParameters(command=sys.executable, args=["math_server.py"])
    async with Client(params) as client:
        # 1. Fetch tools from the MCP server and convert them
        tools = to_openai_tools(await client.list_tools()).tools

        # 2. Send the user message to OpenAI with the tools available
        messages = [
            {"role": "system", "content": "You are a helpful math assistant. Use the available tools to answer math questions."},
            {"role": "user", "content": "What is 5 + 3?"},
        ]
        response = openai_client.responses.create(model=MODEL, input=messages, tools=tools)

        # 3. Execute every tool call the model asked for via MCP

```

```

outputs = []
for item in response.output:
    if item.type == "function_call":
        args = json.loads(item.arguments)
        print(f"LLM called '{item.name}' with args: {args}")
        result = await client.call_tool(item.name, args)
        outputs.append({
            "type": "function_call_output",
            "call_id": item.call_id,
            "output": result.content[0].text if result.content else "",
        })

# 4. Send all tool results back in ONE request for the final answer
if outputs:
    final = openai_client.responses.create(
        model=MODEL, previous_response_id=response.id, input=outputs, tools=tools
    )
    print("Final answer:", final.output_text)
else:
    print("Final answer:", response.output_text)

asyncio.run(main())

```

Key points:

- `client.list_tools()` fetches all MCP tools dynamically — no hardcoding; `to_openai_tools()` converts them in one place
- `previous_response_id` keeps the conversation on OpenAI's side, so you send only the new tool outputs, not the full history
- This works for any tool the LLM picks, not just add
- All tool outputs from one response go back in one request — never one request per tool call
- The model name lives in one place (`OPENAI_MODEL`), so moving to a newer model is a one-line change

Multiple Tool Calls

The model can request several tools in one response (parallel tool calls). The same code handles it: every call is executed, and all results go back together in a single request. Only the question changes:

parallel_tool_calls.py

```
import asyncio
import json
import os
import sys

from dotenv import load_dotenv
from openai import OpenAI

from mcp import Client, StdioServerParameters

load_dotenv() # reads OPENAI_API_KEY from .env
MODEL = os.getenv("OPENAI_MODEL", "gpt-6-luna") # change the model in one place
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def to_openai_tools(mcp_tools):
    """Convert MCP tool definitions to OpenAI Responses API function tools."""
    return [
        {
            "type": "function",
            "name": tool.name,
            "description": tool.description or "",
            "parameters": tool.input_schema, # MCP inputSchema is plain JSON Schema
            "strict": False, # SDK schemas are not strict-mode schemas
        }
        for tool in mcp_tools
    ]

async def main():
```

```

params = StdioServerParameters(command=sys.executable, args=["math_server.py"])
async with Client(params) as client:

    # 1. Fetch tools from the MCP server and convert them
    tools = to_openai_tools((await client.list_tools()).tools)

    # 2. Send the user message to OpenAI with the tools available
    messages = [
        {"role": "system", "content": "You are a helpful math assistant. Use the available tools to answer math questions."},
        {"role": "user", "content": "What is 42 plus 58 and 1 plus 3?"},
    ]
    response = openai_client.responses.create(model=MODEL, input=messages, tools=tools)

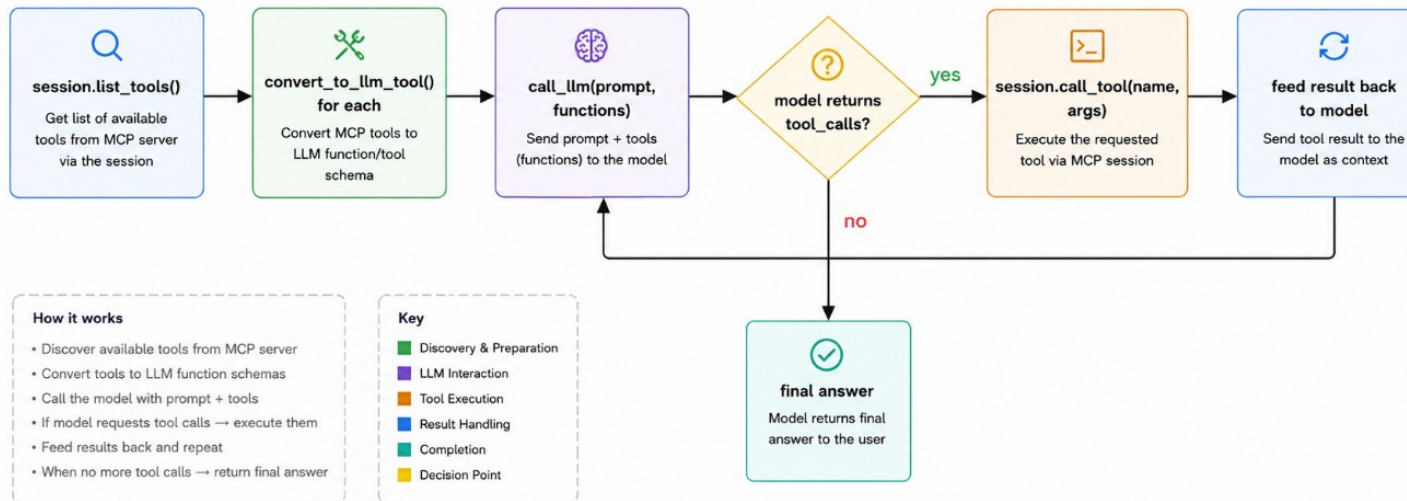
    # 3. Execute every tool call the model asked for via MCP
    outputs = []
    for item in response.output:
        if item.type == "function_call":
            args = json.loads(item.arguments)
            print(f"LLM called '{item.name}' with args: {args}")
            result = await client.call_tool(item.name, args)
            outputs.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": result.content[0].text if result.content else "",
            })

    # 4. Send all tool results back in ONE request for the final answer
    if outputs:
        final = openai_client.responses.create(
            model=MODEL, previous_response_id=response.id, input=outputs, tools=tools
        )
        print("Final answer:", final.output_text)
    else:
        print("Final answer:", response.output_text)

```

```
asyncio.run(main())
```

Multiple Turns with Multiple Tool Calls



When the answer needs a chain of steps (add, then multiply the results), loop until the model stops asking for tools. Each turn sends **only** the new tool outputs together with **previous_response_id**, and **MAX_TURNS** stops a runaway loop.

multi_turn_agent.py

```
import asyncio
import json
import os
import sys

from dotenv import load_dotenv
from openai import OpenAI

from mcp import Client, StdioServerParameters

load_dotenv()
MODEL = os.getenv("OPENAI_MODEL", "gpt-6-luna")
MAX_TURNS = 10 # safety cap for the agent loop
openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

```

def to_openai_tools(mcp_tools):
    return [
        {
            "type": "function",
            "name": tool.name,
            "description": tool.description or "",
            "parameters": tool.input_schema,
            "strict": False,
        }
        for tool in mcp_tools
    ]

async def main():
    params = StdioServerParameters(command=sys.executable, args=["math_server.py"])
    async with Client(params) as client:
        tools = to_openai_tools((await client.list_tools()).tools)

        previous_id = None
        pending = [{"role": "user", "content": "What is 2 plus 8 and 1 plus 3, and then multiply the
results together?"}]

        for turn in range(MAX_TURNS):
            response = openai_client.responses.create(
                model=MODEL, input=pending, tools=tools, previous_response_id=previous_id
            )
            calls = [item for item in response.output if item.type == "function_call"]
            if not calls:
                # no more tool calls → final answer
                print("Final answer:", response.output_text)
                break

            previous_id = response.id
            pending = []
            # send ONLY the new tool outputs next turn
            for item in calls:
                args = json.loads(item.arguments)

```

```

print(f"Turn {turn + 1}: LLM called '{item.name}' with args: {args}")
result = await client.call_tool(item.name, args)
pending.append({
    "type": "function_call_output",
    "call_id": item.call_id,
    "output": result.content[0].text if result.content else "",
})
else:
    print(f"Stopped after {MAX_TURNS} turns without a final answer.")

asyncio.run(main())

```

Expected output (math_server adds 10 on purpose): Turn 1 calls add(2, 8) and add(1, 3) → 20 and 14; Turn 2 calls multiply(20, 14) → 280; then the final answer.

Institute Example with Multiple Tool Calls

SCENARIOS:

1. Find student Arjun Mehta and enroll him in batch 2. (2 calls: search_students → enroll_student)
2. Register a new student Ravi Kumar, email ravi@gmail.com, phone 9111111111. Then enroll him in batch 3." (2 calls: add_student → enroll_student),
3. "Show me who is in batch 1 and also list all students in the Python Full Stack course. (# 2 calls: get_batch_students + get_students_by_course)

Run these prompts with multi_turn_agent.py against the Institute server from the Module 5 demo repository (ServerPrimitivesDemo): change the args in StdioServerParameters to point at its server script, and replace the user message with each scenario.

Letting OpenAI Call the MCP Server Directly

Everything above runs the MCP client in your own code. The Responses API can also connect to a remote MCP server itself: pass a tool of type "mcp" and OpenAI lists and calls the server's tools for you. The server must be reachable from the internet over Streamable HTTP — a local stdio server will not work.

remote_mcp_tool.py

```

import os

from dotenv import load_dotenv
from openai import OpenAI

```

```
load_dotenv()
MODEL = os.getenv("OPENAI_MODEL", "gpt-6-luna")
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# OpenAI connects to the MCP server itself - it must be a public Streamable HTTP URL
response = client.responses.create(
    model=MODEL,
    tools=[{
        "type": "mcp",
        "server_label": "math",
        "server_url": os.getenv("MCP_SERVER_URL", "https://your-server.example.com/mcp"),
        "require_approval": "never",
    }],
    input="What is 5 + 3?",
)
print(response.output_text)
```

Use your own client loop (the earlier examples) for local servers, custom approval logic or any LLM provider; use the built-in **"mcp"** tool for public remote servers when you want OpenAI to manage the calls. Set **require_approval** to **"always"** for tools with side effects.